

CIS 5357

Computing for Data Analytics

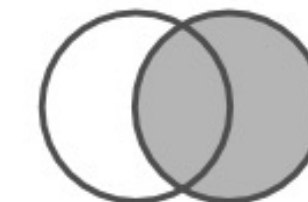
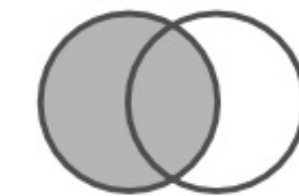
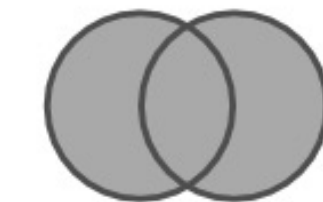
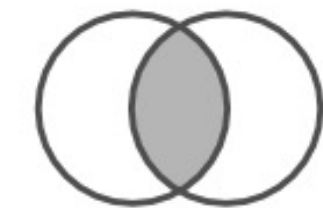
Week 13

Last week

Chapter 7: BRINGING TOGETHER DATASETS

Quick review on joins

1. union `pd.concat()`
2. inner `pd.merge(df1, df2, on=key, how = 'inner')` or `df1.join(df2, how = 'inner')`
3. outer `pd.merge(df1, df2, on=key, how = 'outer')` or `df1.join(df2, how = 'outer')`
4. left `pd.merge(df1, df2, on=key, how = 'left')` or `df1.join(df2, how = 'left')`
5. right `pd.merge(df1, df2, on=key, how = 'right')` or `df1.join(df2, how = 'right')`



Let's practice

AUTHORS

id	author_name	birth_year	death_year
101	Charles Dickens	1812	1870
102	JK Rowling	1965	
103	Agatha Christie	1890	1976
104	JRR Tolkien	1892	1973
...	...	...	...

REVIEWS

book_id	review	rating
1	An incredible book	5
1	Great, although it has some flaws	4
7	Incredible craftsmanship of the author	5
...	...	...

BOOKS

id	author_id	Title	language	published	genre
1	101	<i>A Tale of Two Cities</i>	English	1859	Historical fiction
2	201	<i>Le Petit Prince</i>	French	1943	Fantasy, Children's fiction
3	305	<i>O Alquimista</i>	Portuguese	1988	Fantasy
4	102	<i>Harry Potter and the Philosopher's Stone</i>	English	1997	Fantasy, Children's fiction
5	104	<i>And Then There Were None</i>	English	1939	Mystery
6	402	紅樓夢	Chinese	1791	Family saga
7	104	<i>The Hobbit</i>	English	1937	Fantasy, Children's fiction
...	...	...	...	...	...

Get all books with a review

AUTHORS

id	author_name	birth_year	death_year
101	Charles Dickens	1812	1870
102	JK Rowling	1965	
103	Agatha Christie	1890	1976
104	JRR Tolkien	1892	1973
...	...	...	...

REVIEWS

book_id	review	rating
1	An incredible book	5
1	Great, although it has some flaws	4
7	Incredible craftsmanship of the author	5
...	...	...

BOOKS

id	author_id	Title	language	published	genre
1	101	<i>A Tale of Two Cities</i>	English	1859	Historical fiction
2	201	<i>Le Petit Prince</i>	French	1943	Fantasy, Children's fiction
3	305	<i>O Alquimista</i>	Portuguese	1988	Fantasy
4	102	<i>Harry Potter and the Philosopher's Stone</i>	English	1997	Fantasy, Children's fiction
5	104	<i>And Then There Were None</i>	English	1939	Mystery
6	402	紅樓夢	Chinese	1791	Family saga
7	104	<i>The Hobbit</i>	English	1937	Fantasy, Children's fiction
...	...	...	...	...	...

Get all books with a review

AUTHORS

id	author_name	birth_year	death_year
101	Charles Dickens	1812	1870
102	JK Rowling	1965	
103	Agatha Christie	1890	1976
104	JRR Tolkien	1892	1973
...	...	...	...

REVIEWS

book_id	review	rating
1	An incredible book	5
1	Great, although it has some flaws	4
7	Incredible craftsmanship of the author	5
...	...	...

BOOKS

id	author_id	Title	language	published	genre
1	101	<i>A Tale of Two Cities</i>	English	1859	Historical fiction
2	201	<i>Le Petit Prince</i>	French	1943	Fantasy, Children's fiction
3	305	<i>O Alquimista</i>	Portuguese	1988	Fantasy
4	102	<i>Harry Potter and the Philosopher's Stone</i>	English	1997	Fantasy, Children's fiction
5	104	<i>And Then There Were None</i>	English	1939	Mystery
6	402	紅樓夢	Chinese	1791	Family saga
7	104	<i>The Hobbit</i>	English	1937	Fantasy, Children's fiction
...	...	...	...	...	...

books INNER JOIN review (then LEFT JOIN authors to get names)

List all books & all reviews

AUTHORS

id	author_name	birth_year	death_year
101	Charles Dickens	1812	1870
102	JK Rowling	1965	
103	Agatha Christie	1890	1976
104	JRR Tolkien	1892	1973
...	...	...	...

REVIEWS

book_id	review	rating
1	An incredible book	5
1	Great, although it has some flaws	4
7	Incredible craftsmanship of the author	5
...	...	...

BOOKS

id	author_id	Title	language	published	genre
1	101	<i>A Tale of Two Cities</i>	English	1859	Historical fiction
2	201	<i>Le Petit Prince</i>	French	1943	Fantasy, Children's fiction
3	305	<i>O Alquimista</i>	Portuguese	1988	Fantasy
4	102	<i>Harry Potter and the Philosopher's Stone</i>	English	1997	Fantasy, Children's fiction
5	104	<i>And Then There Were None</i>	English	1939	Mystery
6	402	紅樓夢	Chinese	1791	Family saga
7	104	<i>The Hobbit</i>	English	1937	Fantasy, Children's fiction
...	...	...	...	...	...

List all books & all reviews

AUTHORS

id	author_name	birth_year	death_year
101	Charles Dickens	1812	1870
102	JK Rowling	1965	
103	Agatha Christie	1890	1976
104	JRR Tolkien	1892	1973
...	...	...	...

REVIEWS

book_id	review	rating
1	An incredible book	5
1	Great, although it has some flaws	4
7	Incredible craftsmanship of the author	5
...	...	...

BOOKS

id	author_id	Title	language	published	genre
1	101	<i>A Tale of Two Cities</i>	English	1859	Historical fiction
2	201	<i>Le Petit Prince</i>	French	1943	Fantasy, Children's fiction
3	305	<i>O Alquimista</i>	Portuguese	1988	Fantasy
4	102	<i>Harry Potter and the Philosopher's Stone</i>	English	1997	Fantasy, Children's fiction
5	104	<i>And Then There Were None</i>	English	1939	Mystery
6	402	紅樓夢	Chinese	1791	Family saga
7	104	<i>The Hobbit</i>	English	1937	Fantasy, Children's fiction
...	...	...	...	...	...

books OUTER JOIN reviews

All books with their reviews (if any)

AUTHORS

id	author_name	birth_year	death_year
101	Charles Dickens	1812	1870
102	JK Rowling	1965	
103	Agatha Christie	1890	1976
104	JRR Tolkien	1892	1973
...	...	...	...

REVIEWS

book_id	review	rating
1	An incredible book	5
1	Great, although it has some flaws	4
7	Incredible craftsmanship of the author	5
...	...	...

BOOKS

id	author_id	Title	language	published	genre
1	101	<i>A Tale of Two Cities</i>	English	1859	Historical fiction
2	201	<i>Le Petit Prince</i>	French	1943	Fantasy, Children's fiction
3	305	<i>O Alquimista</i>	Portuguese	1988	Fantasy
4	102	<i>Harry Potter and the Philosopher's Stone</i>	English	1997	Fantasy, Children's fiction
5	104	<i>And Then There Were None</i>	English	1939	Mystery
6	402	紅樓夢	Chinese	1791	Family saga
7	104	<i>The Hobbit</i>	English	1937	Fantasy, Children's fiction
...	...	...	...	...	...

All books with their reviews (if any)

AUTHORS

id	author_name	birth_year	death_year
101	Charles Dickens	1812	1870
102	JK Rowling	1965	
103	Agatha Christie	1890	1976
104	JRR Tolkien	1892	1973
...	...	...	...

REVIEWS

book_id	review	rating
1	An incredible book	5
1	Great, although it has some flaws	4
7	Incredible craftsmanship of the author	5
...	...	...

BOOKS

id	author_id	Title	language	published	genre
1	101	<i>A Tale of Two Cities</i>	English	1859	Historical fiction
2	201	<i>Le Petit Prince</i>	French	1943	Fantasy, Children's fiction
3	305	<i>O Alquimista</i>	Portuguese	1988	Fantasy
4	102	<i>Harry Potter and the Philosopher's Stone</i>	English	1997	Fantasy, Children's fiction
5	104	<i>And Then There Were None</i>	English	1939	Mystery
6	402	紅樓夢	Chinese	1791	Family saga
7	104	<i>The Hobbit</i>	English	1937	Fantasy, Children's fiction
...	...	...	...	...	...

reviews RIGHT JOIN books

books LEFT JOIN reviews

Coding assignment: return analysis

- How many returns occurred?
- What percentage of total orders are returned?
- Which region has the highest percentage of returns?
- Anything else interesting about the return information?

How many returns occurred?

```
file_path = 'Superstore.xlsx'  
returns = pd.read_excel(file_path, sheet_name='Returns')  
returns.head()
```

✓ 0.1s

	Returned	Order ID
0	Yes	US-2019-100762
1	Yes	US-2019-100762
2	Yes	US-2019-100762
3	Yes	US-2019-100762
4	Yes	US-2019-100867

```
unq_returns = returns.drop_duplicates()  
unq_returns.info()
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>  
Int64Index: 296 entries, 0 to 798  
Data columns (total 2 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   Returned    296 non-null    object  
1   Order ID    296 non-null    object  
dtypes: object(2)  
memory usage: 6.9+ KB
```

Note duplicate Order IDs

296 orders returned

What % of orders are returned?

```
orders = pd.read_excel(file_path, sheet_name='Orders')
region_orders = orders[['Region', 'Order ID']]
uniq_region_orders = region_orders.drop_duplicates()
orders_returned = pd.merge(uniq_region_orders, uniq_returns, on = 'Order ID', how = 'left', validate = 'one_to_one')
orders_returned.Returned.isnull().value_counts()
```

✓ 1.2s

```
True      4815
False      296
Name: Returned, dtype: int64
```

```
orders_returned.Returned.isnull().value_counts(normalize = True)
```

✓ 0.0s

```
True      0.942086
False     0.057914
Name: Returned, dtype: float64
```

~5.8% of orders have a return

Could also join then dedupe

```
file_path = 'Superstore.xlsx'
returns = pd.read_excel(file_path, sheet_name='Returns')
orders = pd.read_excel(file_path, sheet_name='Orders')
orders_returned = (pd.merge(orders[['Region', 'Order ID']], returns, on = 'Order ID', how = 'left')).drop_duplicates()
orders_returned.Returned.isnull().value_counts(normalize = True)
```

✓ 1.1s

```
True      0.942086
False     0.057914
Name: Returned, dtype: float64
```

Region with highest % of returns?

```
# filter for the returns in each region, then divide by orders in each region
orders_returned[orders_returned.Returned == "Yes"].Region.value_counts()/orders_returned.Region.value_counts()
```

✓ 0.0s

West	0.115596
East	0.029831
Central	0.033079
South	0.029197

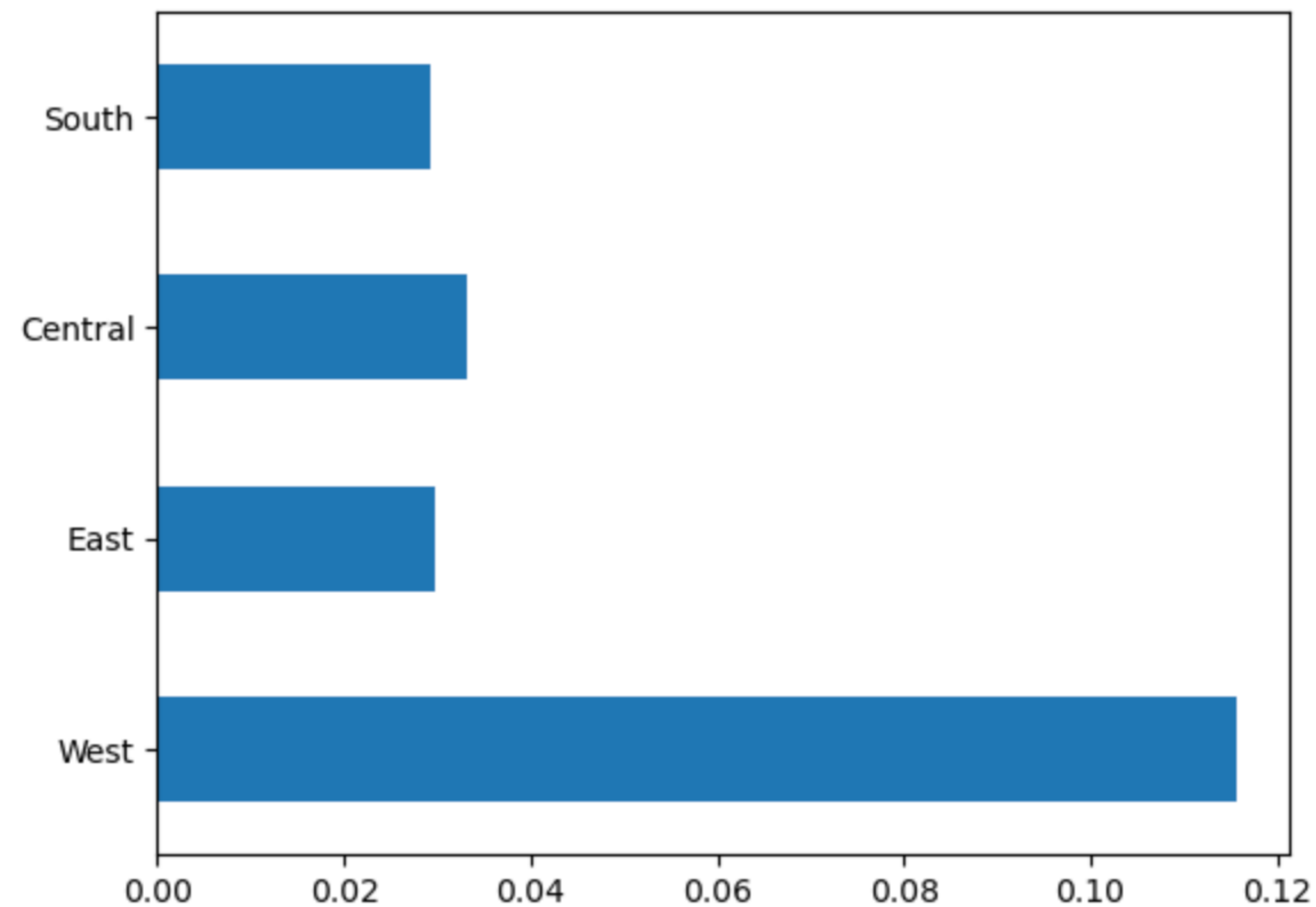
Name: Region, dtype: float64

West region has the most returns: 11.5%

Let's visualize the data!

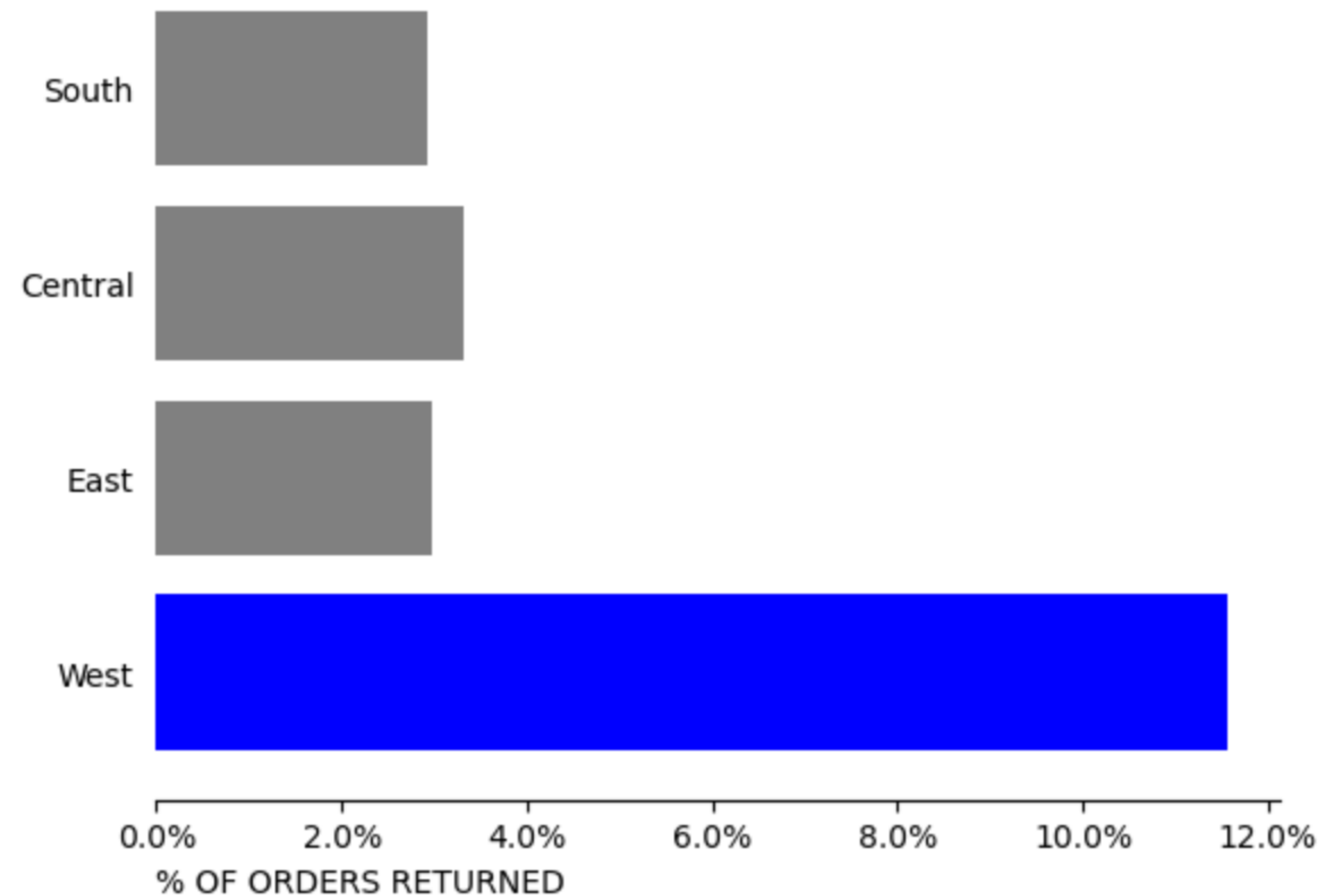
```
✓ (orders_returned[orders_returned.Returned == "Yes"].Region.value_counts()/orders_returned.Region.value_counts()).plot(kind = 'barh')  
] ✓ 0.0s
```

<Axes: >



Let's emphasize the takeaway!

West region has the highest % of orders returned

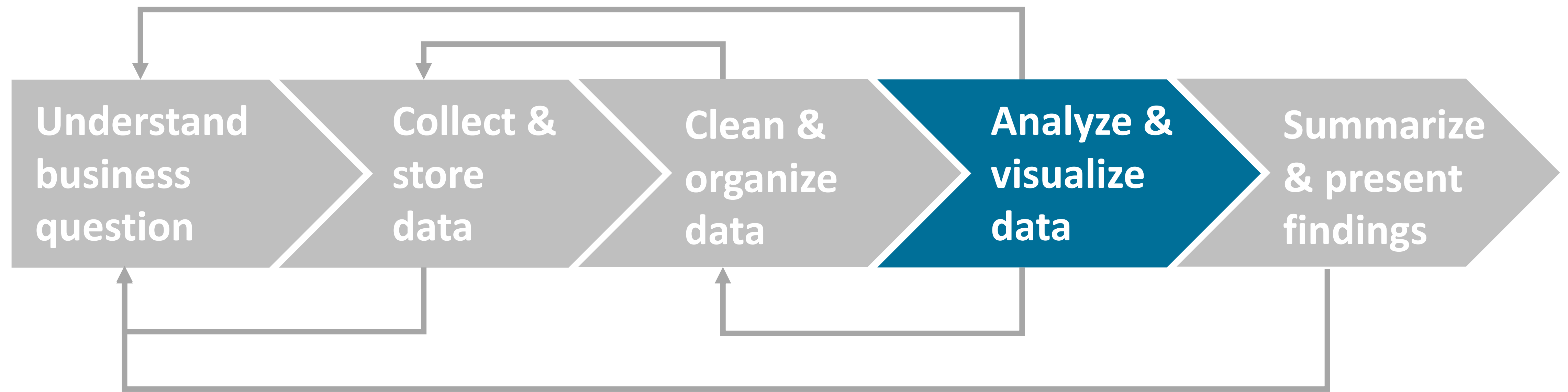


This week

Chapter 8: AGGREGATION

Data Analytics

the process of analyzing raw data in order to draw out meaningful, actionable insights, which are then used to inform and drive business decisions



Download code to follow along



Module 13

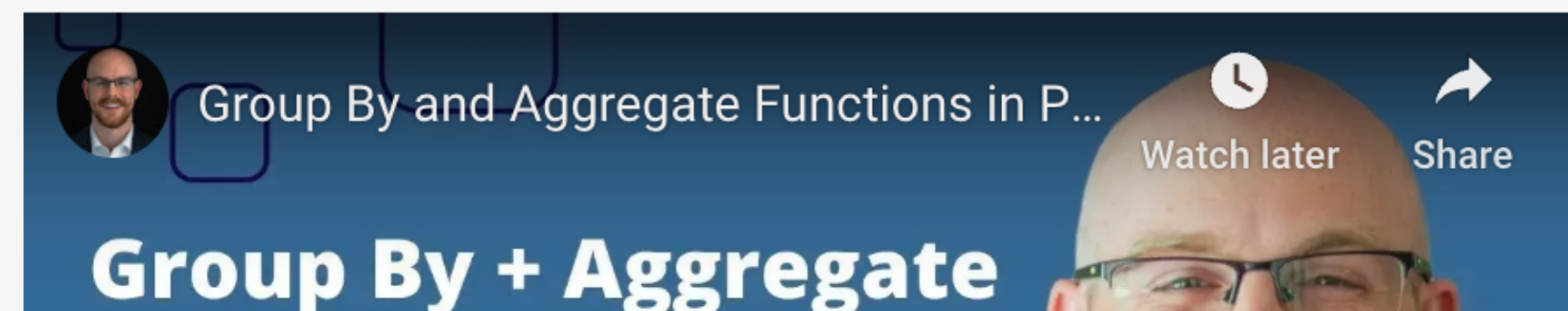
Read and View

Read

- Textbook chapter 8
- [Pandas: group by](#) 
- [Pandas: agg\(\) function](#) 
- [Time series / date functionality](#)  & [resample\(\)](#) 
- [Pandas cheatsheet](#) 
- [Practice code for lecture](#) 

View

[Group by and aggregate functions in pandas](#) 



We've already seen aggregation

- `value_counts()`
- `describe()`
- `min()`
- `max()`
- `mean()`
- `median()`
- `sum()`
- `std()`

Counting aggregations available

- **size**: returns the number of rows in each group including NaN missing values
- **count()**: returns the number of rows in each group not including NaN missing values
- **nunique()**: returns the number of unique values in each group, ignoring any NaN missing values

Let's consider *df_orders*

```
# count the number of orders for each main  
df_orders.MAIN.value_counts()
```

✓ 0.0s

I0 693847

I3 693502

I1 590685

I2 183158

I4 114447

Name: MAIN, dtype: int64

totals to
2,275,639

Let's consider *df_orders*

```
# count the number of orders for each main  
df_orders.MAIN.value_counts()
```

✓ 0.0s

```
I0    693847  
I3    693502  
I1    590685  
I2    183158  
I4    114447  
Name: MAIN, dtype: int64
```

totals to
2,275,639

```
# the number of rows with a main  
df_orders.MAIN.size
```

✓ 0.0s

2387224

Let's consider *df_orders*

```
# count the number of orders for each main  
df_orders.MAIN.value_counts()
```

✓ 0.0s

```
I0    693847  
I3    693502  
I1    590685  
I2    183158  
I4    114447  
Name: MAIN, dtype: int64
```

totals to
2,275,639

```
# the number of rows with a main  
df_orders.MAIN.size
```

✓ 0.0s

2387224

```
# count the number of orders with a main  
df_orders.MAIN.count()
```

✓ 0.0s

2275639

```
# a unique count of the mains that were ordered  
df_orders.MAIN.nunique()
```

✓ 0.0s

5

Multiple ways to group data

```
# count the number of orders for each main  
df_orders.MAIN.value_counts()
```

✓ 0.0s

I0 693847

I3 693502

I1 590685

I2 183158

I4 114447

Name: MAIN, dtype: int64

`value_counts()` groups by the specified column name

We can also use `groupby()`

The benefit of ***groupby()*** is that it allows us to create groups over more than one column. Let us, for example, calculate the average number of daily orders at each restaurant

```
( df_summarized_orders.groupby('RESTAURANT_NAME')  
  .NUM_ORDERS.mean() )
```

RESTAURANT_NAME	
Bryant Park	871.436782
Columbia	687.150685
Flatiron	772.556164
Midtown	891.136986
NYU	1180.438356
Upper East Side	774.650704
Upper West Side	770.372603
Williamsburg	862.120548

Name: NUM_ORDERS, dtype: float64

1. Select the data in **df_summarized_orders**
2. Pass **RESTAURANT_NAME** column to specify we want to separate the dataset by that column.
3. Next, we select **NUM_ORDERS** as the one we want to do calculations over.
4. Finally, we apply the ***mean()*** function to find the average of that column.

Notes about *groupby()*

The output is a series, not a table, but we can use `reset_index()` to create a dataframe

```
df_summarized_orders.groupby('RESTAURANT_NAME').NUM_ORDERS.mean().reset_index()
```

✓ 0.0s

	RESTAURANT_NAME	NUM_ORDERS
0	Bryant Park	871.436782
1	Columbia	687.150685
2	Flatiron	772.556164
3	Midtown	891.136986
4	NYU	1180.438356
5	Upper East Side	774.650704
6	Upper West Side	770.372603
7	Williamsburg	862.120548

Grouping by a series

```
df_orders.groupby(df_orders.DRINKS > 0).COOKIES.mean()
```

✓ 0.0s

DRINKS

False 0.259543

True 0.258149

Name: COOKIES, dtype: float64

Here the text inside the **groupby()** function, `df_order.DRINKS > 0`, is simply a series that contains True if an order has a drink, and False otherwise and pandas automatically groups by the result of that series.

Group by multiple columns

pass a list containing the column names we want to group by

```
df_orders.groupby(['RESTAURANT_NAME', 'TYPE']).ORDER_ID.count()
```

RESTAURANT_NAME	TYPE	
Bryant Park	DELIVERY	15613
	IN_STORE	171494
	PICKUP	40338
Columbia	DELIVERY	25247
	IN_STORE	182603
	PICKUP	42960
Flatiron	DELIVERY	28859
	IN_STORE	204607
	PICKUP	48517
Midtown	DELIVERY	22380
	IN_STORE	244980
	PICKUP	57905
NYU	DELIVERY	43310

note the square brackets here, as the input needs to be a list

Unstack()

The *unstack()* function creates a crosstab

```
df_orders.groupby(['RESTAURANT_NAME', 'TYPE']).ORDER_ID.count().unstack()
```

✓ 0.3s

	TYPE	DELIVERY	IN_STORE	PICKUP
RESTAURANT_NAME				
	Bryant Park	15613	171494	40338
	Columbia	25247	182603	42960
	Flatiron	28859	204607	48517
	Midtown	22380	244980	57905
	NYU	43310	314832	72718
	Upper East Side	52080	180605	42316
	Upper West Side	53337	184588	43261
	Williamsburg	31822	229427	53425

The second group by is now the column headings

Fun fact: Cross tabs can be graphed!

The *unstack()* function creates a crosstab

```
df_orders.groupby(['RESTAURANT_NAME', 'TYPE']).ORDER_ID.count().unstack()
```

✓ 0.3s

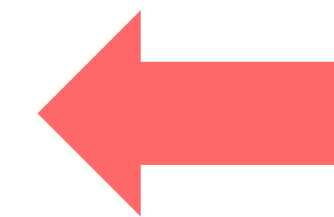
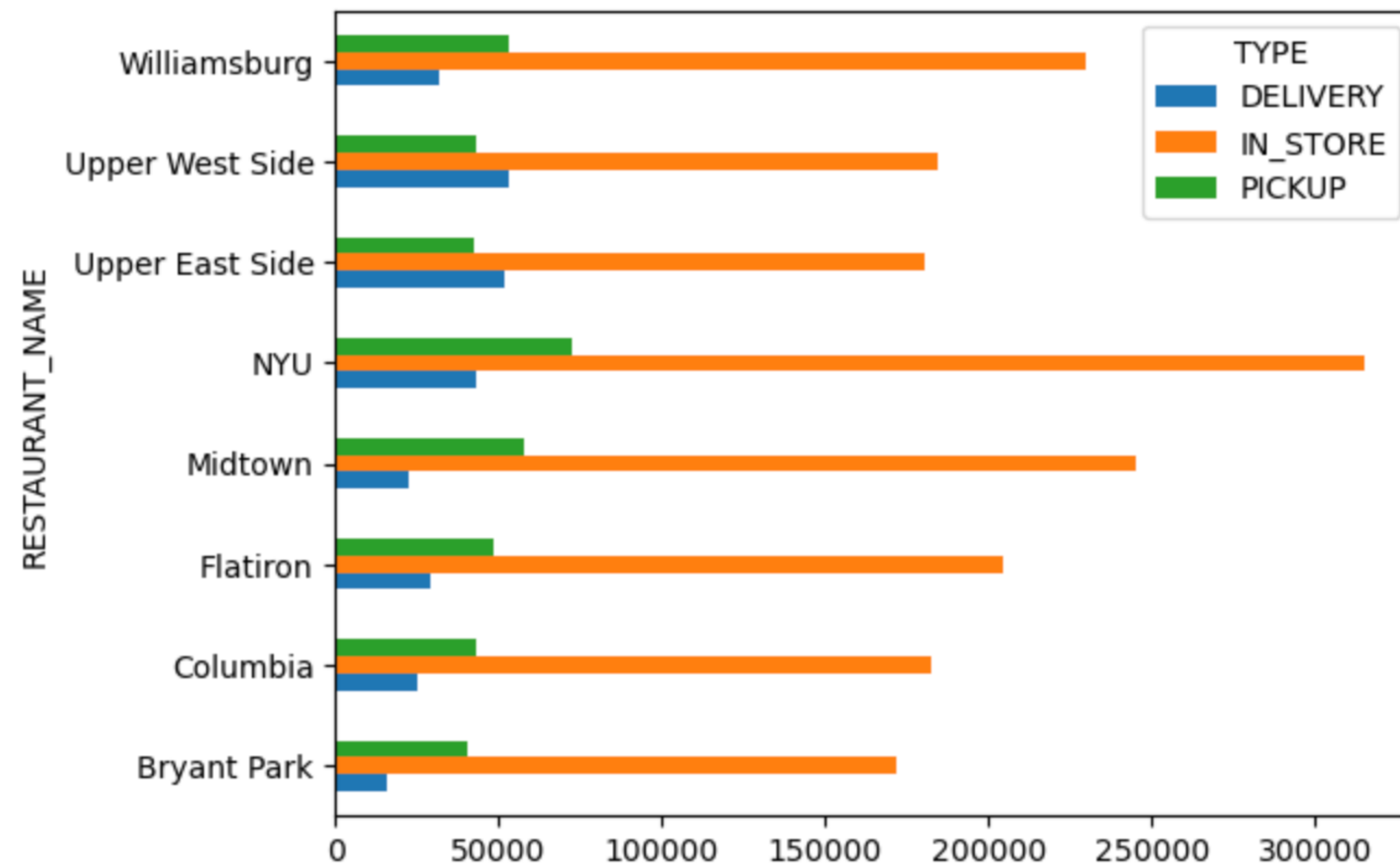
	TYPE	DELIVERY	IN_STORE	PICKUP
RESTAURANT_NAME				
Bryant Park		15613	171494	40338
Columbia		25247	182603	42960
Flatiron		28859	204607	48517
Midtown		22380	244980	57905
NYU		43310	314832	72718
Upper East Side		52080	180605	42316
Upper West Side		53337	184588	43261
Williamsburg		31822	229427	53425

Fun fact: Cross tabs can be graphed!

```
df_orders.groupby(['RESTAURANT_NAME', 'TYPE']).ORDER_ID.count().unstack().plot(kind = 'barh')
```

✓ 0.5s

<Axes: ylabel='RESTAURANT_NAME'>



Columns become separate series in the chart

Aggregate multiple columns

We can also select multiple columns after the groupby()

```
# aggregate more than one column  
df_orders.groupby('RESTAURANT_NAME')[['DRINKS', 'COOKIES']].mean()
```

✓ 0.1s

	DRINKS	COOKIES
RESTAURANT_NAME		
Bryant Park	0.098138	0.261294
Columbia	0.066572	0.259049
Flatiron	0.097637	0.259161
Midtown	0.126128	0.260683
NYU	0.075769	0.258898
Upper East Side	0.118145	0.257603
Upper West Side	0.097032	0.260475
Williamsburg	0.096446	0.258750

note the square brackets here, as the input needs to be a list

What will this yield?

```
df_orders.groupby(['RESTAURANT_NAME', 'TYPE'])[['DRINKS', 'COOKIES']].mean().unstack()
```

✓ 0.2s

What will this yield?

```
df_orders.groupby(['RESTAURANT_NAME', 'TYPE'])[['DRINKS', 'COOKIES']].mean().unstack()
```

✓ 0.2s

TYPE	DRINKS			COOKIES		
	DELIVERY	IN_STORE	PICKUP	DELIVERY	IN_STORE	PICKUP
RESTAURANT_NAME						
Bryant Park	0.107090	0.097251	0.098443	0.261769	0.261653	0.259582
Columbia	0.064483	0.066248	0.069181	0.263239	0.258906	0.257193
Flatiron	0.101078	0.097543	0.095987	0.265186	0.258843	0.256920
Midtown	0.140036	0.124733	0.126656	0.263807	0.260801	0.258976
NYU	0.072108	0.076596	0.074369	0.255114	0.258894	0.261173
Upper East Side	0.120392	0.117156	0.119600	0.262097	0.257390	0.252978
Upper West Side	0.095600	0.097726	0.095837	0.258151	0.260981	0.261182
Williamsburg	0.098611	0.096148	0.096434	0.261737	0.258862	0.256490

Challenge 1: crosstab drinks & cookies

Using the *df_orders* table, create a crosstab with the number of drinks as the rows and the number of cookies as the columns

Challenge 1: crosstab drinks & cookies

Using the *df_orders* table, create a crosstab with the number of drinks as the rows and the number of cookies as the columns

We'll resume in
5 minutes...

Challenge 1: crosstab drinks & cookies

Using the *df_orders* table, create a crosstab with the number of drinks as the rows and the number of cookies as the columns

We'll resume in
4 minutes...

Challenge 1: crosstab drinks & cookies

Using the *df_orders* table, create a crosstab with the number of drinks as the rows and the number of cookies as the columns

We'll resume in
3 minutes...

Challenge 1: crosstab drinks & cookies

Using the *df_orders* table, create a crosstab with the number of drinks as the rows and the number of cookies as the columns

We'll resume in
2 minutes...

Challenge 1: crosstab drinks & cookies

Using the *df_orders* table, create a crosstab with the number of drinks as the rows and the number of cookies as the columns

We'll resume in
1 minute...

Challenge 1: crosstab drinks & cookies

Using the *df_orders* table, create a crosstab with the number of drinks as the rows and the number of cookies as the columns

One possible solution

```
df_orders.groupby(['DRINKS', 'COOKIES']).ORDER_ID.count().unstack()
```

✓ 0.4s

COOKIES	0.0	1.0	2.0
DRINKS			
0.0	1760520.0	307666.0	131642.0
1.0	123777.0	21450.0	9196.0
2.0	21574.0	3737.0	1603.0
3.0	3774.0	644.0	286.0
4.0	722.0	129.0	64.0
5.0	114.0	18.0	7.0
6.0	28.0	4.0	3.0
7.0	23.0	5.0	1.0
8.0	14.0	3.0	3.0
9.0	15.0	4.0	2.0
10.0	20.0	1.0	NaN
11.0	18.0	1.0	2.0
12.0	14.0	5.0	2.0
13.0	27.0	4.0	2.0

Different functions on each column

Suppose we want to create a dataframe with the following data for each restaurant:

- The average number of drinks in each order: `DRINKS.mean()`
- The number of orders with a main: `MAIN.count()`

Brute force method

```
# perform different calculations for each column; brute force option: merge separate dataframes
df_cookies_avg = df_orders.groupby('RESTAURANT_NAME').COOKIES.mean()
df_main_cnt = df_orders.groupby('RESTAURANT_NAME').MAIN.count()
df_combined = pd.merge(df_cookies_avg, df_main_cnt, on = 'RESTAURANT_NAME', how = 'outer' )
df_combined
```

] ✓ 0.2s

	COOKIES	MAIN
RESTAURANT_NAME		
Bryant Park	0.261294	216767
Columbia	0.259049	239406
Flatiron	0.259161	268909
Midtown	0.260683	309502
NYU	0.258898	411253
Upper East Side	0.257603	261957
Upper West Side	0.260475	268023
Williamsburg	0.258750	299822

Could use the agg() function also

```
df_orders.groupby('RESTAURANT_NAME').agg(AVG_COOKIES = ('COOKIES', 'mean'),  
NUM_W_MAIN = ('MAIN', 'count'))
```

✓ 0.2s

RESTAURANT_NAME	AVG_COOKIES	NUM_W_MAIN
Bryant Park	0.261294	216767
Columbia	0.259049	239406
Flatiron	0.259161	268909
Midtown	0.260683	309502
NYU	0.258898	411253
Upper East Side	0.257603	261957
Upper West Side	0.260475	268023
Williamsburg	0.258750	299822

- The function take **one argument for every column** you would like
- Each argument should be the **name of the output column**, and it should be **equal to two terms in ()**
 1. the first should be the input column in the original table (e.g., 'COOKIES'),
 2. the second should be the name of the function to apply, as a string (e.g., 'mean').

Can also pass a dictionary in `agg()`

```
df_orders.groupby('RESTAURANT_NAME').agg({'COOKIES': ['min', 'max', 'mean', 'count'], 'MAIN': ['count']})
```

✓ 0.2s

	COOKIES				MAIN
	min	max	mean	count	count
RESTAURANT_NAME					
Bryant Park	0.0	2.0	0.261294	227445	216767
Columbia	0.0	2.0	0.259049	250810	239406
Flatiron	0.0	2.0	0.259161	281983	268909
Midtown	0.0	2.0	0.260683	325265	309502
NYU	0.0	2.0	0.258898	430860	411253
Upper East Side	0.0	2.0	0.257603	275001	261957
Upper West Side	0.0	2.0	0.260475	281186	268023
Williamsburg	0.0	2.0	0.258750	314674	299822

note the difference in counts—why?
count does not include NaN missing values

Let's talk date functions

Robust **datetime** functionality in pandas can be accessed using **dt**

- *dt.weekday* returns the day of the week, from zero (Monday) to six (Sunday)
- *dt.day_name()*, which returns the name of the day of the week (note, confusingly, that it requires parentheses whereas weekday does not).
- *dt.weekofyear* and *dt.dayofyear* return the number of weeks and days since the start of the year respectively
- *dt.quarter* returns the quarter in which the date falls.
- *dt.normalize()* returns the timestamp, but with the time set to midnight.
- *dt.to_period('X')* converts to period specified, 'Y' for year, 'M' for month, 'W' for week, 'D' for day, etc.
- ...many more available in documentation

Group using date functions

```
# can also use dt functions (in Chapter 6) to group  
df_orders.groupby([df_orders.DATETIME.dt.to_period('D')]).COOKIES.mean().reset_index().head()
```

✓ 0.0s

groups by the day

	DATETIME	COOKIES
0	2018-01-01	0.258356
1	2018-01-02	0.261985
2	2018-01-03	0.269168
3	2018-01-04	0.245617
4	2018-01-05	0.266439

A special case for dates

Pandas has a function called ***resample()***, which works exactly like *groupby()*, but for dates. It requires the datetime column as the index of the dataframe and takes in an argument to indicate the level of aggregation:

- T—minutes
- H—hours
- D—days
- W—weeks
- M—months
- Y—years

Let's consider an example

Average cookies per order per day

```
df_orders.set_index('DATETIME').resample('D').COOKIES.mean().reset_index().head()
```

✓ 2.8s

	DATETIME	COOKIES
0	2018-01-01	0.258356
1	2018-01-02	0.261985
2	2018-01-03	0.269168
3	2018-01-04	0.245617
4	2018-01-05	0.266439

1. Select the data in **df_order**
2. Set the index of the dataframe to the **DATETIME** column.
3. Apply the **resample()** function—specify that we want to group by day using **D**
4. Select **COOKIES** as the one we want to do calculations over.
5. Then, we apply the **mean()** function to find the average of that column
6. Use **reset_index()** to create a dataframe
7. Apply **head()** to look at the first 5 rows

Challenge 2: plot monthly orders

Using the *df_orders* table, create a visual that shows the number of orders each month.

Challenge 2: plot monthly orders

Using the *df_orders* table, create a visual that shows the number of orders each month.

We'll resume in
8 minutes...

Challenge 2: plot monthly orders

Using the *df_orders* table, create a visual that shows the number of orders each month.

We'll resume in
7 minutes...

Challenge 2: plot monthly orders

Using the *df_orders* table, create a visual that shows the number of orders each month.

We'll resume in
6 minutes...

Challenge 2: plot monthly orders

Using the *df_orders* table, create a visual that shows the number of orders each month.

We'll resume in
5 minutes...

Challenge 2: plot monthly orders

Using the *df_orders* table, create a visual that shows the number of orders each month.

We'll resume in
4 minutes...

Challenge 2: plot monthly orders

Using the *df_orders* table, create a visual that shows the number of orders each month.

We'll resume in
3 minutes...

Challenge 2: plot monthly orders

Using the *df_orders* table, create a visual that shows the number of orders each month.

We'll resume in
2 minutes...

Challenge 2: plot monthly orders

Using the *df_orders* table, create a visual that shows the number of orders each month.

We'll resume in
1 minute...

Challenge 2: plot monthly orders

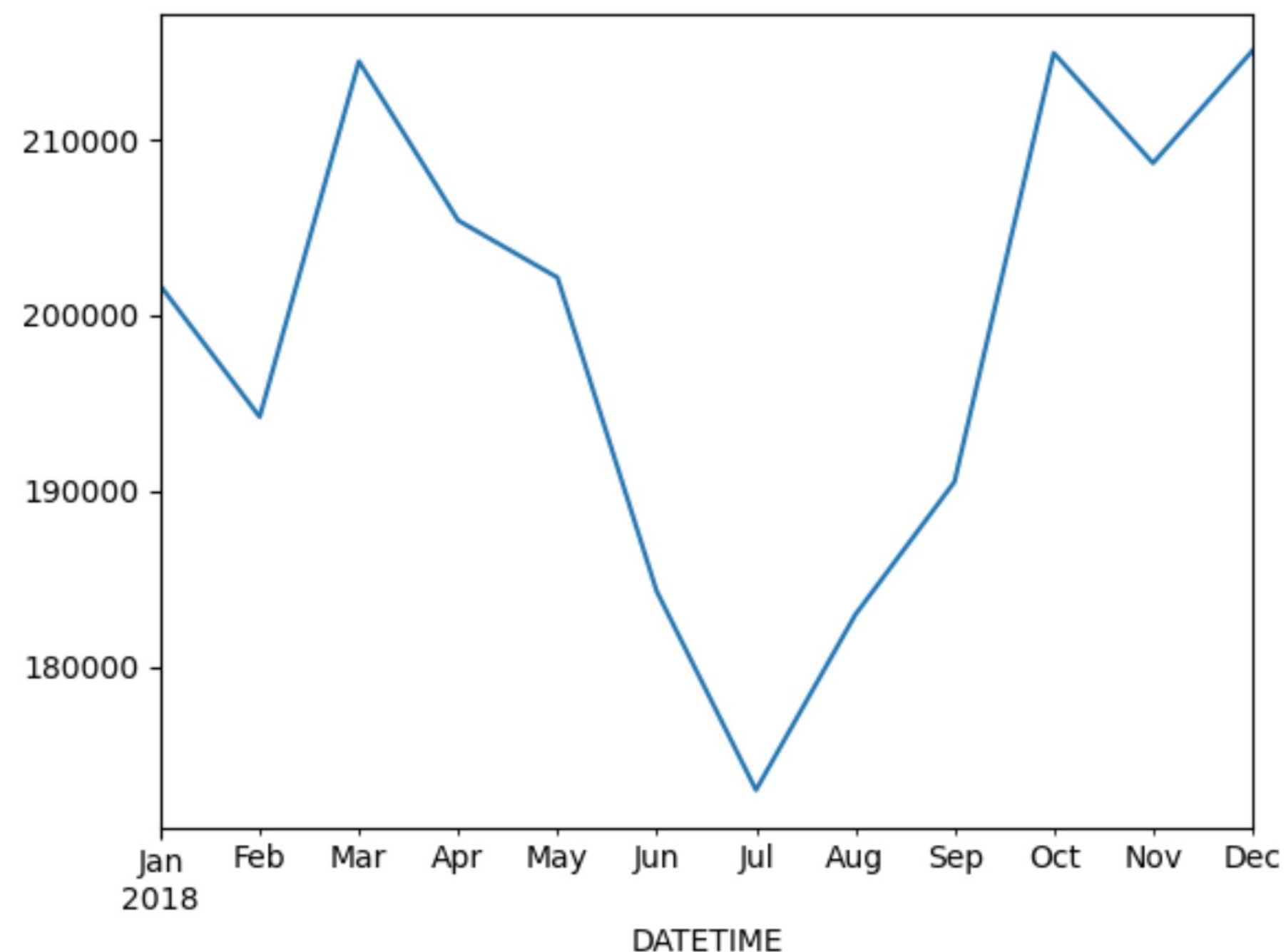
Using the *df_orders* table, create a visual that shows the number of orders each month.

Possible solutions using resample

```
df_orders.set_index(df_orders.DATETIME).resample('M').size().plot(x = "DATETIME", y="ORDER_ID")
```

✓ 2.5s

<Axes: xlabel='DATETIME'>



These also work for the data:

```
df_orders.set_index(df_orders.DATETIME)
        .resample('M')
        .ORDER_ID
        .count()
        .reset_index()
        .plot(x = "DATETIME", y="ORDER_ID")
```

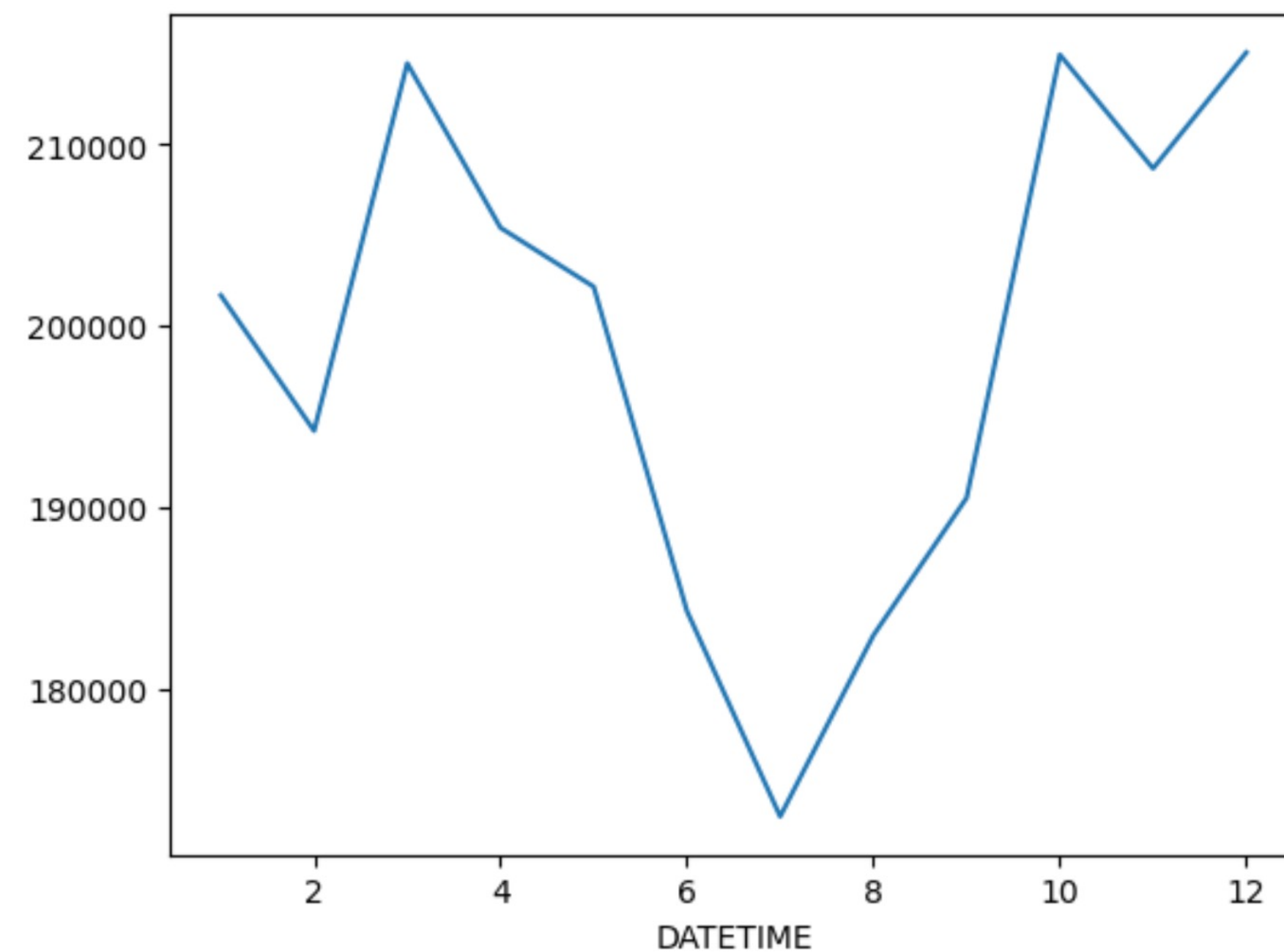
```
df_orders.set_index(df_orders.DATETIME)
        .resample('M')
        .ORDER_ID
        .nunique()
        .reset_index()
        .plot(x = "DATETIME", y="ORDER_ID")
```

Or use date functions to group

```
df_orders.groupby([df_orders.DATETIME.dt.month]).ORDER_ID.count().plot()
```

✓ 0.2s

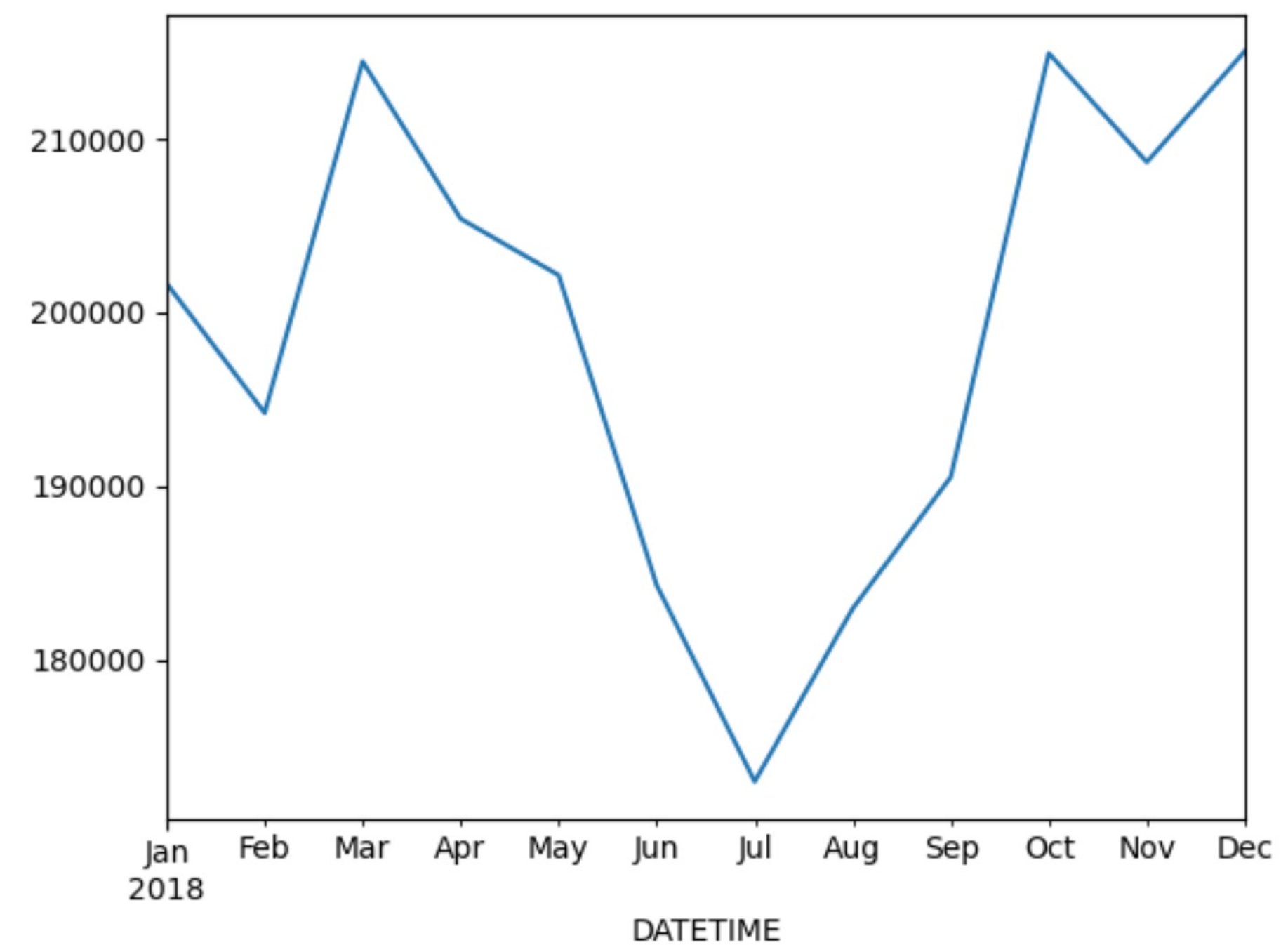
<Axes: xlabel='DATETIME'>



```
df_orders.groupby([df_orders.DATETIME.dt.to_period('M')]).ORDER_ID.count().plot()
```

✓ 0.2s

<Axes: xlabel='DATETIME'>



Challenge 3: orders by location

Using the *df_orders* table, create a visual that shows the number of orders each month for each restaurant location—include the user-friendly restaurant names.

Challenge 3: orders by location

Using the *df_orders* table, create a visual that shows the number of orders each month for each restaurant location—include the user-friendly restaurant names.

We'll resume in
5 minutes...

Challenge 3: orders by location

Using the *df_orders* table, create a visual that shows the number of orders each month for each restaurant location—include the user-friendly restaurant names.

We'll resume in
4 minutes...

Challenge 3: orders by location

Using the *df_orders* table, create a visual that shows the number of orders each month for each restaurant location—include the user-friendly restaurant names.

We'll resume in
3 minutes...

Challenge 3: orders by location

Using the *df_orders* table, create a visual that shows the number of orders each month for each restaurant location—include the user-friendly restaurant names.

We'll resume in
2 minutes...

Challenge 3: orders by location

Using the *df_orders* table, create a visual that shows the number of orders each month for each restaurant location—include the user-friendly restaurant names.

We'll resume in
1 minute...

Challenge 3: orders by location

Using the *df_orders* table, create a visual that shows the number of orders each month for each restaurant location—include the user-friendly restaurant names.

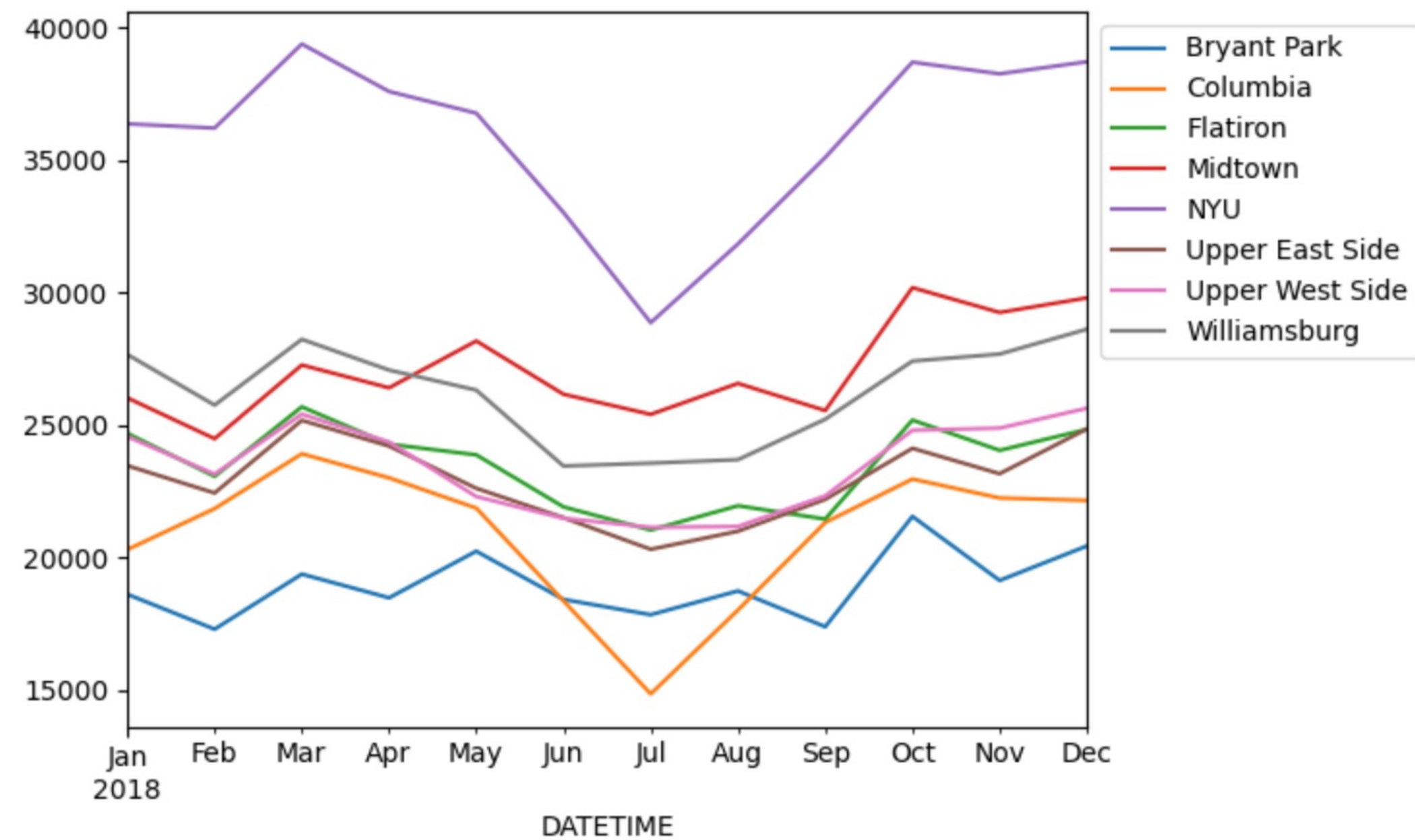
One possible solution

```
(df_orders.groupby([df_orders.DATETIME.dt.to_period('M'),df_orders.RESTAURANT_NAME])  
  .ORDER_ID  
  .count()  
  .unstack()  
  .plot()  
  .legend(bbox_to_anchor=(1,1)))
```

sets the position of the legend

✓ 0.4s

<matplotlib.legend.Legend at 0x164e16890>



Next week

Dig **analysis**

- launching new drinks
- delivery menu items
- staffing solutions
- customer analysis

Final **exam** overview

Meet on **Zoom**